

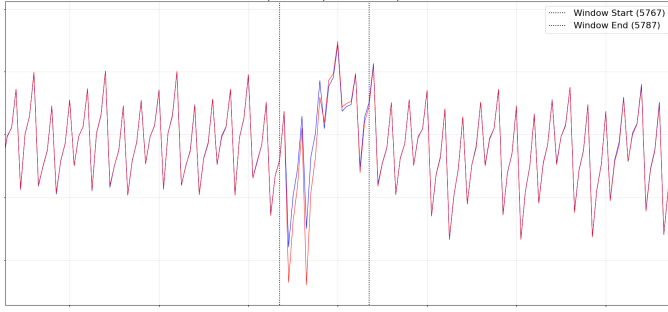


Fig. 9: Isolated multiplication operation in unprotected traces

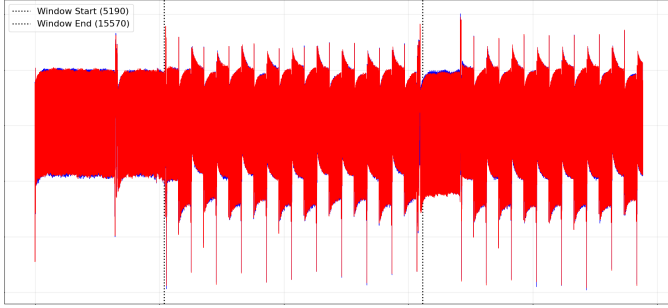


Fig. 10: Isolated neuron computation with isolated multiplication operation

APPENDIX

ADDITIONAL WINDOW SELECTION EXPERIMENTS

Several additional trace-window configurations were evaluated during attempts to achieve the highest success rate in recovering internal information from unprotected implementation using the CPA.

A. Localized Small-Window Analysis

Initial experiments attempted to isolate leakage using very small trace windows containing approximately 20 samples around suspected arithmetic operations. These arithmetic operations were isolated to make them easier to identify with NOPs.

These windows were selected using visual comparison of overlaid traces generated from:

- all-zero inputs,
- maximum positive input,
- maximum negative input.

The objective was to identify regions where power consumption visibly differed depending on processed values (Figure 9).

Despite visible waveform differences, localized CPA analysis on these windows did not produce stable or statistically dominant correlation peaks.

Additional experiments attempted to isolate computations of:

- the first hidden neuron,
- selected multiplication operations.

Large numbers of NOP instructions were inserted between operations to improve separation between neuron computations and to isolate multiplication operation.

Although several experiments produced localized correlation peaks, the strongest and most stable results were consistently obtained using full-trace analysis rather than highly isolated windows.

This suggests that leakage behavior is distributed across multiple overlapping operations rather than being confined to a single short arithmetic region.

PROJECT STRUCTURE AND IMPLEMENTATION DETAILS

This appendix provides a comprehensive overview of the research project demonstrating defense mechanisms against Side-Channel Attacks (SCA) on embedded neural networks.

B. Project Overview

The research follows a three-stage pipeline:

- 1) **Python Development:** Designing, training, and quantizing a baseline neural network.
- 2) **Embedded C Implementation:** Porting the quantized model to C for ARM Cortex-M4 (CWLITEARM).
- 3) **Side-Channel Analysis:** Evaluating security through Correlation Power Analysis (CPA) to compare unprotected vs. protected implementations.

C. Project Directory Structure

Below is the organized file structure for the research repository:

- **compiled_for_CWLITEARM/** Contains the C source code, network headers, and makefiles specifically configured for the ChipWhisperer hardware target.
- **cpa_traces_unprot/** Includes 5,000 pre-captured power traces, input vectors, and the Python-based CPA attack script for the baseline model.
- **protected_traces/** Includes 5,000 pre-captured power traces and the corresponding attack scripts for models using binary code encoding.
- **nn_in_python/** Reference implementations used for training and testing the quantization logic before embedded deployment.
- **protected_nn_with_lut_MULT_ADD_ARRAYS/TABLES** folders which contains the network with proposed protection implementations. They can be tested locally, and these folders contains subfolder with all related to compilation and deployment on ChipWhisperer-Lite.
- **Utility Scripts** Includes `transform_dataset.py` for binary encoding and `capture_traces.ipynb` for the hardware workflow.

D. Practical Usage Guide

1) Compilation and Deployment

To build the firmware for the ARM Cortex target, navigate to the source directory and execute the build system:

```
cd compiled_for_CWLITEARM_unprotected_symmetric
```

```
make clean && make
PLATFORM=CWLITEARM
```

This process generates the .hex file required for flashing the target device.

2) Binary Encoding and LUT Generation

The protection mechanism relies on pre-computed tables. These are generated and the dataset is transformed using:

```
gcc generate_all.c -o generate_all
&& ./generate_all
python transform_dataset.py
```

Inputs are mapped to protected binary codes.

3) Execution of Correlation Power Analysis (CPA)

To evaluate the effectiveness of the implemented protections, run the following analysis scripts:

For Baseline:

```
python cpa_for_unprotected.py
```

For Protected:

```
python cpa_for_protected.py
```